

# DART XML 파서 규칙과 전체 검증 결과

| 항목      | 내용                                                                       |
|---------|--------------------------------------------------------------------------|
| 문서 상태   | DART XML 전체 파싱 검증 완료본                                                    |
| 작성일     | 2026-08-14                                                               |
| 대상 데이터  | 대회 제공 데이터의 <b>DART_XML</b> 원문                                            |
| 대상 문서 수 | 3,147개                                                                   |
| 선행 문서   | <b>04_dart_xml_structure_profiling.md</b>                                |
| 주요 구현   | <b>DartXmlDisclosureParser</b> , <b>XmlParsingValidationBatchService</b> |

## 1. 문서 목적

이 문서는 DART XML 원문 파서가 현재 어떤 규칙으로 문서를 구조화하는지와 전체 데이터 검증 결과를 기록한다.

구체적으로 다음 내용을 다룬다.

- XML 파일을 안전하게 읽고 문법 오류를 보정하는 방법
- **DOCUMENT-NAME**, **SECTION-N**, **TITLE**, **P**, **TABLE**, **IMAGE**, **PGBRK** 처리 규칙
- 파싱 결과 객체의 구조
- 소규모 단위 테스트와 전체 3,147개 배치 검증 방식
- DB 및 기존 구조 조사 결과와의 교차 검증 결과
- 현재 파서가 보장하는 범위와 아직 보장하지 않는 범위
- 파싱 결과 DB 적재 전 다음 작업

이 문서는 파서 구현의 현재 기준선이다. 파서 규칙을 변경할 때에는 코드, 테스트, 이 문서와 검증 결과를 함께 갱신한다.

## 2. 작업 배경

선행 구조 조사에서 DART XML 3,147개에 다음 특징이 있음을 확인했다.

- **SECTION-1**부터 **SECTION-4**까지 존재한다.
- 표가 매우 많고 중첩 표의 최대 깊이는 3이다.
- 전체 **P**의 대부분이 표 안에 존재한다.
- 표 셀에는 **ROWSPAN**, **COLSPAN**, 내부 문단과 이미지가 포함될 수 있다.
- 명시적인 줄바꿈 태그 대신 **PGBRK**가 많이 사용된다.
- 이미지 바이너리는 없고 XML 안에는 파일명과 캡션 같은 메타데이터만 있다.
- 일부 원문에는 XML에서 그대로 허용되지 않는 **&**, **<**, 속성 따옴표가 있다.

따라서 단순히 태그의 모든 텍스트를 하나의 문자열로 합치는 방식은 사용할 수 없다. 현재 파서는 장·절, 문단, 표, 중첩 표, 이미지 메타데이터와 원문 행 위치를 보존하는 구조를 만든다.

## 3. 처리 범위

### 3.1 포함 범위

- DB의 `content_format = DART_XML` 문서
- XML 파일 경로와 확장자 검증
- XML 보안 설정
- 읽기 과정의 제한적 문법 보정
- 문서명 추출
- `SECTION-N` 계층 구성
- 제목과 일반 문단 구분
- 표의 행·셀·병합 속성 보존
- 중첩 표 보존
- 이미지 메타데이터 보존
- 페이지 구분 보존
- XML 원문 행 위치 보존
- 문서별 성공·경고·실패와 구조 지표 CSV 출력

### 3.2 제외 범위

현재 단계에서는 다음 작업을 하지 않는다.

- 파싱 결과 엔티티 생성과 DB 적재
- `disclosure_documents.parse_status` 변경
- 검색 청크 생성
- 짧은 문단 병합
- 검색용 표 텍스트 직렬화
- 공시 유형별 중요 사실 추출
- 금액·날짜·비율 계산
- 이미지 바이너리 복원
- HTML 1,472개 파싱
- PDF 3개 파싱
- 공시 문장의 금융 의미 검증

## 4. 전체 처리 흐름

```

disclosure_documents의 DART_XML 문서 조회
↓
DB 상대경로와 실제 파일 경로 검증
↓
DartXmlSourceFileValidator
↓
DartXmlSanitizingReader
↓
보안 설정이 적용된 StAX XMLStreamReader
↓
DartXmlDisclosureParser
↓
ParsedDisclosureDocument
├─ preambleBlocks
└─ sections
  
```

```

    └─ blocks
    └─ children
↓
XmlParsingValidationMetricsCollector
↓
문서별 검증 결과 CSV

```

파서는 XML 전체 트리를 한 번에 메모리에 올리는 DOM 방식이 아니라 이벤트를 순차적으로 읽는 StAX 방식을 사용한다.

## 5. 파싱 결과 모델

### 5.1 문서와 섹션

```

ParsedDisclosureDocument
├─ fileName
├─ documentName
├─ preambleBlocks
└─ sections: List<ParsedDisclosureSection>
    └─ level
    └─ order
    └─ title
    └─ sourceLineStart
    └─ sourceLineEnd
    └─ blocks
    └─ children

```

`preambleBlocks`는 아직 어떤 `SECTION-N`에도 들어가지 않은 문서 앞부분의 블록이다.

`children`은 섹션의 부모·자식 관계를 보존한다.

### 5.2 본문 블록

```

ParsedDisclosureBlock
├─ type
│   └─ HEADING
│   └─ PARAGRAPH
│   └─ TABLE
│   └─ IMAGE
│   └─ PAGE_BREAK
├─ order
├─ content
├─ table
├─ image
├─ sourceLineStart
└─ sourceLineEnd

```

블록 유형에 따라 사용하는 값이 다르다.

| 블록 유형      | 사용하는 값          |
|------------|-----------------|
| HEADING    | content         |
| PARAGRAPH  | content         |
| TABLE      | table           |
| IMAGE      | image           |
| PAGE_BREAK | 별도 내용 없이 위치만 사용 |

### 5.3 표

```
ParsedDisclosureTable
└─ rows: List<ParsedDisclosureTableRow>
    └─ cells: List<ParsedDisclosureTableCell>
        │ type: HEADER | DATA
        │ rowspan
        │ colspan
        │ text
        │ nestedTables
        └─ images
```

표를 단일 문자열로 평탄화하지 않고 실제 **TABLE** → **TR** → **TH/TD** 관계를 보존한다.

### 5.4 이미지

```
ParsedDisclosureImage
└─ fileName
└─ caption
└─ width
└─ height
└─ alignment
└─ sourceLineStart
└─ sourceLineEnd
```

이 객체는 이미지 바이너리가 아니다. XML에 기록된 이미지 파일명과 캡션 등 메타데이터만 나타낸다.

## 6. 공통 XML 입력 규칙

### 6.1 파일 검증

**DartXmlSourceFileValidator**는 파싱 전에 다음 조건을 확인한다.

- 경로가 null이 아니다.
- 실제 파일이 존재한다.
- 심볼릭 링크가 아니다.
- 일반 파일이다.

- 읽기 가능하다.
- 파일명이 `.xml`로 끝난다.

배치 검증 단계에서는 추가로 다음 조건을 확인한다.

- DB의 `manifest_path`에서 계산한 공시 폴더 안에 파일이 있다.
- 파일명이 공시 폴더 밖으로 탈출하는 경로가 아니다.
- DB의 `normalized_relative_path`와 실제 파일 상대경로가 일치한다.

## 6.2 XML 보안 설정

`DartXmlInputFactoryProvider`는 호출할 때마다 새로운 `XMLInputFactory`를 생성한다.

필수 설정은 다음과 같다.

| 설정              | 값     | 목적               |
|-----------------|-------|------------------|
| DTD             | 비활성화  | 외부 선언과 엔티티 공격 방지 |
| 외부 엔티티          | 비활성화  | 로컬 파일·네트워크 접근 방지 |
| Namespace aware | 활성화   | XML 이름 처리 일관성    |
| Coalescing      | 활성화   | 인접 문자와 CDATA 결합  |
| XML Resolver    | 항상 예외 | 외부 XML 리소스 접근 차단 |

필수 보안 속성을 XML 구현체가 지원하지 않으면 파싱을 계속하지 않고 실패시킨다.

## 6.3 원문 보정

`DartXmlSanitizingReader`는 원본 파일을 수정하지 않는다. 파서가 파일을 읽는 스트림에서만 잘못된 문자를 보정한다.

현재 보정 규칙은 다음과 같다.

| 원문 문제                                     | 읽기 시 처리                      |
|-------------------------------------------|------------------------------|
| XML 엔티티가 아닌 단독 <code>&amp;</code>         | <code>&amp;amp;</code> 로 변환  |
| 일반 문장에 사용된 단독 <code>&lt;</code>           | <code>&amp;lt;</code> 로 변환   |
| 본문 약어처럼 사용된 <code>&lt;CG&gt;</code> 등의 표현 | 일반 텍스트로 변환                   |
| 속성값 시작 직후 중복 따옴표                          | 중복 따옴표 제거                    |
| 속성값中间的 잘못된 따옴표                            | <code>&amp;quot;</code> 로 변환 |

기본 XML 엔티티와 숫자 엔티티는 그대로 유지한다.

```
& &lt; &gt; &quot; &apos;
&#123; &#x1F;
```

구조 조사 기준으로 같은 3,147개 문서에서 다음 보정이 발견됐다.

| 보정 종류      | 보정이 있었던 문서 | 전체 보정 횟수 |
|------------|------------|----------|
| 단독 &       | 1,620개     | 84,075회  |
| 일반 텍스트의 <  | 1,352개     | 42,699회  |
| 잘못된 속성 따옴표 | 79개        | 662회     |

이 문서들도 전체 파싱 검증에서 모두 성공했다.

## 7. 태그별 파싱 규칙

### 7.1 DOCUMENT-NAME

- 문서에서 처음 만난 DOCUMENT-NAME만 문서명으로 사용한다.
- 내부 문자와 CDATA를 합친다.
- 연속 공백을 하나로 줄이고 앞뒤 공백을 제거한다.
- 정규화 결과가 비어 있으면 null로 처리한다.

### 7.2 SECTION-N

섹션 단계는 다음 정규식으로 판별한다.

```
^SECTION-([1-9]\d*)$
```

따라서 현재 데이터의 SECTION-1부터 SECTION-4뿐 아니라 향후 SECTION-5 이상도 모델에 담을 수 있다.

규칙은 다음과 같다.

- 시작 태그를 만나면 SectionBuilder를 스택에 넣는다.
- 종료 태그를 만나면 가장 안쪽 섹션을 완성한다.
- 상위 섹션이 있으면 상위 섹션의 children에 넣는다.
- 상위 섹션이 없으면 문서의 최상위 sections에 넣는다.
- 단계, 문서 등장 순서, 제목, 시작 행과 종료 행을 보존한다.
- 종료되지 않은 섹션이 남으면 문서 파싱을 실패시킨다.

### 7.3 TITLE

- 섹션 안에서 처음 만난 유효한 TITLE은 해당 섹션의 대표 제목으로 사용한다.
- 같은 섹션의 두 번째 이후 TITLE은 HEADING 블록으로 보존한다.
- 섹션 밖의 TITLE도 HEADING 블록으로 preambleBlocks에 보존한다.
- 연속 공백은 하나로 줄이고 앞뒤 공백은 제거한다.

구조 조사에서 TITLE 내부 P는 0건이었다. 현재 데이터에서는 제목과 문단의 중복 수집 문제가 발생하지 않는다.

향후 TITLE 내부에 P가 들어간 새 데이터가 추가된다면 중복 수집 방지 규칙과 회귀 테스트를 별도로 추가해야 한다.

### 7.4 일반 P

- 표 밖의 P는 **PARAGRAPH** 블록으로 만든다.
- 내부의 강조 태그 등은 별도 객체로 만들지 않지만 문자 내용은 이어서 수집한다.
- 일반 문단은 모든 연속 공백과 개행을 하나의 공백으로 정규화한다.
- 비어 있는 문단은 결과에 추가하지 않는다.
- 문단의 시작 행과 종료 행을 보존한다.

짧은 문단을 이전·다음 문단과 합치는 작업은 파서가 수행하지 않는다. 문단 정규화 또는 검색 청크 생성 단계가 별도로 담당한다.

## 7.5 표 **TABLE**, **TR**, **TH**, **TD**

표는 별도 스택으로 처리한다.

### 표 계층

- 최상위 **TABLE**은 **TABLE** 본문 블록이 된다.
- 표를 읽는 중 새 **TABLE**을 만나면 중첩 표로 처리한다.
- 중첩 표는 반드시 부모 표의 열린 셀 안에 있어야 한다.
- 중첩 표는 부모 셀의 **nestedTables**에 추가한다.
- 중첩 표 내용을 부모 셀 텍스트에 평탄화하지 않는다.

### 행과 셀

- **TR** 하나는 **ParsedDisclosureTableRow** 하나가 된다.
- **TH**는 **HEADER**, **TD**는 **DATA**로 구분한다.
- 행과 셀의 등장 순서는 0부터 시작하는 인덱스로 보존한다.
- 시작 행과 종료 행을 보존한다.
- 열린 행·셀이 정상적으로 닫히지 않으면 파싱을 실패시킨다.

### 셀 병합 속성

- **ROWSPAN**, **COLSPAN**이 없으면 기본값은 1이다.
- 속성값은 1 이상의 정수여야 한다.
- 0, 음수 또는 숫자가 아닌 값은 허용하지 않는다.
- 현재 단계에서는 병합 속성을 보존하지만 물리 셀을 논리 격자로 확장하지 않는다.

### 셀 텍스트와 줄바꿈

- 표 내부 P는 별도 **PARAGRAPH** 블록으로 만들지 않는다.
- P가 끝날 때 셀 텍스트에 줄바꿈을 추가한다.
- 한 줄 안의 연속 공백은 하나로 줄인다.
- XML 들여쓰기에서 발생한 빈 줄은 제거한다.
- 서로 다른 P 사이의 줄바꿈은 **\n**으로 보존한다.
- 빈 셀 또는 중첩 표만 있는 셀의 텍스트는 **null**일 수 있다.

## 7.6 **IMAGE**, **IMG**, **IMG-CAPTION**

이미지는 실제 파일을 읽지 않고 다음 XML 메타데이터만 보존한다.

- **IMG** 내부 파일명
- **WIDTH**
- **HEIGHT**
- **ALIGN**
- **IMG-CAPTION** 내부 캡션
- XML 시작 행과 종료 행

규칙은 다음과 같다.

- 일반 본문의 이미지는 **IMAGE** 블록으로 만든다.
- 표 셀 안의 이미지는 셀의 **images** 목록에 넣는다.
- 표 안의 이미지를 셀 텍스트에 섞지 않는다.
- **WIDTH**, **HEIGHT**는 값이 있을 때 1 이상의 정수여야 한다.
- **ALIGN**은 대문자로 정규화한다.
- 캡션이 . 하나뿐이면 의미 없는 값으로 보고 **null**로 처리한다.
- 파일명과 캡션이 모두 없으면 유효한 이미지로 만들지 않는다.
- **IMAGE** 중첩은 허용하지 않는다.

이미지 파일명 예시가 **1.jpg**여도 실제 이미지 파일이 데이터에 없을 수 있다. 현재 결과는 이미지 존재와 캡션을 검색 근거로 활용할 수 있게 메타데이터만 보존한다.

## 7.7 PGBRK

**PGBRK**의 위치에 따라 다르게 처리한다.

| 위치         | 처리                      |
|------------|-------------------------|
| 일반 본문      | <b>PAGE_BREAK</b> 블록 생성 |
| 일반 문단 내부   | 문단 문자 구분 추가             |
| 제목 내부      | 공백 추가                   |
| 표의 열린 셀 내부 | 셀 줄바꿈 추가                |
| 표 안이지만 셀 밖 | 레이아웃용으로 보고 무시           |
| 이미지 내부     | 무시                      |

같은 블록 목록에 **PAGE\_BREAK**가 연속으로 나타나면 하나만 보존한다.

## 7.8 기타 태그와 이벤트

- 태그 이름은 대문자로 정규화해 비교한다.
- 문자 이벤트와 CDATA 이벤트를 모두 수집한다.
- 일반 문단이나 셀 내부의 장식·강조 태그는 별도 객체로 만들지 않는다.
- 장식 태그 안의 문자 내용은 현재 수집 중인 문단·제목·셀에 포함한다.
- XML 주석 등 현재 모델에 대응하지 않는 이벤트는 무시한다.

# 8. 순서와 원문 위치 규칙

## 8.1 문서 순서



섹션과 최상위 본문 블록은 문서에서 발견된 순서대로 **order**를 부여한다.

표는 별도의 **table order**를 가진다. 중첩 표를 포함해 문서에서 표가 시작된 순서를 나타낸다.

## 8.2 원문 행

다음 객체는 시작 행과 종료 행을 보존한다.

- 섹션
- 본문 블록
- 표
- 표 행
- 표 셀
- 이미지

원문 행은 이후 주장과 공시 원문 근거를 연결하는 데 사용한다.

보정 Reader는 한 입력 행을 한 출력 행으로 유지하므로 문자를 치환해도 전체 행 번호는 유지된다.

## 9. 파싱 실패 규칙

다음과 같은 구조 오류는 해당 문서를 실패로 처리한다.

- 시작 없이 종료된 섹션
- 종료되지 않은 섹션
- 종료되지 않은 표
- **TR** 밖의 **TH** 또는 **TD**
- 이전 행·셀이 닫히기 전에 새 행·셀 시작
- 열린 셀이 있는 상태에서 행 종료
- 셀 밖의 중첩 표
- 셀 밖의 표 내부 이미지
- 중첩된 **IMAGE**
- 종료되지 않은 **IMAGE**
- 잘못된 **ROWSPAN**, **COLSPAN**, 이미지 크기 속성
- XML 문법을 제한적 보정으로 복구할 수 없는 경우

배치 검증은 한 문서의 실패가 다음 문서 검증을 중단시키지 않도록 문서별로 예외를 격리한다.

XML 예외가 있으면 예외 체인에서 **XMLStreamException**을 찾아 행과 열을 CSV에 기록한다. 오류 메시지는 개행을 제거하고 최대 2,000자로 제한한다.

## 10. 단위 테스트 범위

현재 **DartXmlDisclosureParserTest**는 다음 규칙을 검증한다.

1. 문서명과 중첩 섹션 파싱
2. **ROWSPAN**, **COLSPAN**과 중첩 표 파싱
3. 일반 본문 이미지 메타데이터 파싱
4. 표 셀 이미지와 셀 텍스트 분리
5. 일반 본문의 **PGBRK** 순서 보존과 연속 중복 제거
6. 표 셀 내부 **PGBRK**의 줄바꿈 변환

## 7. 일반 문장에 사용된 < 보정

전체 배치 검증이 통과했더라도 대표 문서 테스트는 유지해야 한다. 파서 규칙 변경 시 최소한 위 테스트와 전체 CSV 검증을 다시 수행한다.

# 11. 배치 검증 방식

## 11.1 대상 조회

`XmlParsingValidationBatchService`는 다음 기준으로 문서를 조회한다.

```
content_format = DART_XML
ORDER BY disclosure_document_id ASC
page = 0 이상
limit = 1~500
```

문서를 한 번에 병렬로 파싱하지 않고 페이지 안에서 순차적으로 처리한다.

## 11.2 검증 지표

문서별로 다음 값을 집계한다.

| 지표                                | 의미                                         |
|-----------------------------------|--------------------------------------------|
| <code>section_count</code>        | 모든 단계의 섹션 개수                               |
| <code>max_section_level</code>    | 가장 깊은 <code>SECTION-N</code>               |
| <code>total_block_count</code>    | 문서 앞부분과 섹션의 최상위 블록 수                       |
| <code>heading_count</code>        | 추가 제목 블록 수                                 |
| <code>paragraph_count</code>      | 일반 문단 블록 수                                 |
| <code>page_break_count</code>     | 페이지 구분 블록 수                                |
| <code>table_count</code>          | 최상위 표와 중첩 표를 합친 수                          |
| <code>nested_table_count</code>   | 다른 표 셀 안에 있는 표 수                           |
| <code>table_row_count</code>      | 모든 표 행 수                                   |
| <code>table_cell_count</code>     | <code>TH</code> 와 <code>TD</code> 를 합친 셀 수 |
| <code>image_count</code>          | 본문 이미지와 셀 이미지 수                            |
| <code>text_character_count</code> | 섹션 제목·블록·셀·이미지 캡션 문자 수                     |
| <code>elapsed_millis</code>       | 해당 문서 검증 시간                                |

`total_block_count`에는 중첩 표 자체를 별도 최상위 블록으로 더하지 않는다. 중첩 표는 `table_count`와 `nested_table_count`에 재귀적으로 반영한다.

## 11.3 상태 판정

| 상태      | 조건                                   |
|---------|--------------------------------------|
| SUCCESS | 파싱 성공, 문서명 존재, 블록과 텍스트가 1개 이상        |
| WARNING | 파싱됐지만 문서명·블록·텍스트 중 하나가 없음            |
| FAILED  | 경로 확인, 파일 읽기, 보정, XML 파싱 또는 결과 생성 실패 |

WARNING은 현재 다음 우선순위로 하나의 메시지를 기록한다.

1. DOCUMENT-NAME 없음
2. 구조화된 본문 블록 없음
3. 구조화된 텍스트 없음

## 11.4 CSV 출력

CSV에는 다음 27개 필드를 기록한다.

```
disclosure_document_id
receipt_no
source_group
report_name
file_name
document_role
file_size_bytes
parsed_document_name
section_count
max_section_level
total_block_count
heading_count
paragraph_count
page_break_count
table_count
nested_table_count
table_row_count
table_cell_count
image_count
text_character_count
elapsed_millis
status
warning_message
error_type
error_line
error_column
error_message
```

CSV는 UTF-8로 저장하며 Windows Excel에서도 한글을 올바르게 인식하도록 파일 맨 앞에 UTF-8 BOM을 기록한다.

## 12. 실행 설정

.env 예시는 다음과 같다.

```
FOLIOLENS_VALIDATE_XML_PARSING_ON_STARTUP=true
FOLIOLENS_VALIDATE_XML_PARSING_PAGE=0
FOLIOLENS_VALIDATE_XML_PARSING_LIMIT=500
FOLIOLENS_VALIDATE_XML_PARSING_REPORT_FILE_NAME=xml-parsing-validation-page-000.csv
```

페이지를 진행할 때에는 페이지 번호와 파일명을 함께 변경한다.

```
page=0 → xml-parsing-validation-page-000.csv
page=1 → xml-parsing-validation-page-001.csv
...
page=6 → xml-parsing-validation-page-006.csv
```

Docker 이미지에 코드 변경을 반영해 실행한다.

```
docker compose up --build
```

배치 완료 후 검증 Runner를 반복 실행하지 않으려면 다음 값을 `false`로 되돌린다.

```
FOLIOLENS_VALIDATE_XML_PARSING_ON_STARTUP=false
```

## 13. 전체 검증 결과

### 13.1 페이지별 결과

| 페이지 | 파일                                  | 문서 수  | 성공    | 경고 | 실패 |
|-----|-------------------------------------|-------|-------|----|----|
| 0   | xml-parsing-validation-page-000.csv | 500   | 500   | 0  | 0  |
| 1   | xml-parsing-validation-page-001.csv | 500   | 500   | 0  | 0  |
| 2   | xml-parsing-validation-page-002.csv | 500   | 500   | 0  | 0  |
| 3   | xml-parsing-validation-page-003.csv | 500   | 500   | 0  | 0  |
| 4   | xml-parsing-validation-page-004.csv | 500   | 500   | 0  | 0  |
| 5   | xml-parsing-validation-page-005.csv | 500   | 500   | 0  | 0  |
| 6   | xml-parsing-validation-page-006.csv | 147   | 147   | 0  | 0  |
| 합계  |                                     | 3,147 | 3,147 | 0  | 0  |

### 13.2 공시 그룹

| 공시 그룹     | 문서 수         | 성공률         |
|-----------|--------------|-------------|
| periodic  | 1,466        | 100%        |
| major     | 598          | 100%        |
| holding   | 1,083        | 100%        |
| <b>합계</b> | <b>3,147</b> | <b>100%</b> |

exchange 원문은 현재 HTML로 판별되므로 DART XML 검증 대상에 포함되지 않는다.

### 13.3 문서 역할

| 역할         | 문서 수         | 성공률         |
|------------|--------------|-------------|
| MAIN       | 2,732        | 100%        |
| ATTACHMENT | 415          | 100%        |
| <b>합계</b>  | <b>3,147</b> | <b>100%</b> |

### 13.4 CSV 무결성

| 확인 항목        | 결과     |
|--------------|--------|
| CSV 파일       | 7개     |
| 전체 행         | 3,147개 |
| 고유 문서 ID     | 3,147개 |
| 중복 문서 ID     | 0개     |
| 필수값 누락       | 0개     |
| 예상하지 않은 상태   | 0개     |
| 헤더 불일치       | 0개     |
| UTF-8 BOM 누락 | 0개     |

## 14. DB 및 구조 조사 교차 검증

### 14.1 DB 문서 집합 비교

DB의 content\_format = DART\_XML 문서 ID와 7개 CSV의 문서 ID를 비교했다.

| 항목              | 결과     |
|-----------------|--------|
| DB의 DART XML    | 3,147개 |
| CSV의 고유 문서      | 3,147개 |
| CSV에서 누락된 DB 문서 | 0개     |

| 항목               | 결과 |
|------------------|----|
| DB에 없는 추가 CSV 문서 | 0개 |

따라서 전체 DART XML 문서가 페이지 중복이나 누락 없이 한 번씩 검증됐다.

## 14.2 구조 조사 결과 비교

동일한 3,147개 문서를 `xml-additional-structure` 결과와 문서 ID로 연결해 비교했다.

| 비교 항목    | 불일치 문서 |
|----------|--------|
| 파일 크기    | 0개     |
| 문서명      | 0개     |
| 전체 섹션 수  | 0개     |
| 최대 섹션 단계 | 0개     |
| 전체 표 수   | 0개     |
| 중첩 표 수   | 0개     |
| 표 행 수    | 0개     |
| 표 셀 수    | 0개     |

구조 조사에서는 `TH`와 `TD`를 따로 집계하므로 셀 수는 다음 기준으로 비교했다.

```

파싱 table_cell_count
= 구조 조사 table_header_count
+ 구조 조사 table_cell_count

```

이 값도 모든 문서에서 일치했다.

이는 단순히 XML 예외가 발생하지 않았다는 의미를 넘어, 섹션과 표의 핵심 구조가 파싱 결과에 보존됐다는 근거다.

## 15. 검증 불변조건

다음 문제가 있는 행을 전체 CSV에서 확인했으며 모두 0개였다.

| 검증 조건                                             | 위반 문서 |
|---------------------------------------------------|-------|
| 음수 크기·카운트·처리시간                                    | 0개    |
| <code>max_section_level &gt; section_count</code> | 0개    |
| <code>nested_table_count &gt; table_count</code>  | 0개    |
| 성공했지만 문서명이 없음                                     | 0개    |
| 성공했지만 블록이 0개                                      | 0개    |

| 검증 조건             | 위반 문서 |
|-------------------|-------|
| 성공했지만 텍스트가 0자     | 0개    |
| 실패했지만 오류 정보가 없음   | 0개    |
| 성공·경고인데 오류 정보가 있음 | 0개    |

일반 문단이 0개인 문서는 5개였다. 모두 주요사항보고서의 표 중심 문서이며 표·셀·텍스트가 정상적으로 존재하므로 실패로 보지 않는다.

## 16. 구조 지표 범위

| 지표         | 전체 문서 최댓값 |
|------------|-----------|
| 섹션 수       | 70        |
| 섹션 단계      | 4         |
| 최상위 본문 블록  | 7,544     |
| 전체 표       | 4,872     |
| 전체 표 행     | 23,685    |
| 전체 표 셀     | 98,995    |
| 구조화 텍스트 문자 | 1,112,786 |

큰 값은 주로 정기공시에서 발생한다. 기존 구조 조사 값과 일치하므로 현재 검증에서는 중복 파싱으로 부풀려진 값으로 판단하지 않는다.

## 17. 성능 결과

문서별 `elapsed_millis` 합계를 기준으로 한 결과다. Docker 시작 시간과 페이지 사이의 사용자 대기시간은 포함하지 않는다.

| 항목          | 결과       |
|-------------|----------|
| 문서별 처리시간 합계 | 약 22.79분 |
| 평균          | 434.5ms  |
| 중앙값         | 186ms    |
| 90백분위       | 1,138ms  |
| 95백분위       | 1,506ms  |
| 99백분위       | 2,496ms  |
| 최대          | 6,476ms  |

공시 그룹별 결과는 다음과 같다.

| 공시 그룹    | 문서 수  | 평균      | 합계       |
|----------|-------|---------|----------|
| major    | 598   | 104.5ms | 약 1.04분  |
| holding  | 1,083 | 136.7ms | 약 2.47분  |
| periodic | 1,466 | 789.0ms | 약 19.28분 |

가장 오래 걸린 문서는 약 30MB의 대형 정기공시였으며 6.476초가 걸렸다. 사용자 요청 시 실행되는 기능이 아니라 적재 전 내부 배치라는 점을 고려하면 현재 성능은 허용 가능하다.

## 18. 완료 판단

### 18.1 완료된 것

- DB의 모든 DART XML 3,147개 조회
- 실제 파일 경로 검증
- 제한적 XML 문법 보정
- 보안 StAX 파싱
- **SECTION-N** 일반화
- 제목·문단·표·중첩 표 구조화
- 표 행·셀과 병합 속성 보존
- 이미지 메타데이터 보존
- 페이지 구분 처리
- 원문 행 위치 보존
- 문서별 구조 지표 생성
- CSV 한글과 열 정렬 정상화
- 전체 성공·중복 없음·누락 없음 확인
- 기존 구조 조사와 핵심 구조 일치 확인

### 18.2 이번 결과만으로 보장하지 않는 것

- 각 문장의 금융 의미가 정확하다는 보장
- 공시 유형별 중요 필드 추출 정확도
- 표의 헤더와 데이터 행 의미 연결
- 병합 셀을 반영한 완성된 논리 격자
- 짧은 문단 병합과 검색 청크 품질
- 원문 근거가 DB 엔티티로 저장됐다는 보장
- HTML 및 PDF 원문 지원

전체 배치 성공은 구조 파서가 모든 XML을 처리할 수 있다는 강한 근거지만, 금융 사실 추출과 질의응답 정확도를 대신 검증하지는 않는다.

## 19. DB 상태 주의사항

**XmlParsingValidationBatchRunner**는 검증 전용이다.

다음 값을 변경하지 않는다.

- **disclosure\_documents.parse\_status**



- `parser_name`
- `parser_version`
- `parse_error_message`
- `parsed_at`

따라서 전체 검증 후에도 DB의 XML 문서가 **PENDING**인 것은 정상이다. 실제 적재 서비스가 파싱 결과를 트랜잭션으로 저장할 때 상태를 변경해야 한다.

## 20. 다음 작업

XML 파서에 대한 추가 무작위 구조 조사보다 파싱 결과 저장 단계로 이동한다.

권장 순서는 다음과 같다.

1. 대표 회귀 문서 fixture 고정
2. 파서 이름과 규칙 버전 결정
3. `disclosure_sections` 저장 모델 확정
4. `disclosure_content_blocks` 저장 모델 확정
5. 표·행·셀·중첩 표 저장 방식 확정
6. 이미지 메타데이터 저장 여부 확정
7. `ParsedDisclosureDocument`에서 엔티티로 변환하는 Mapper 구현
8. 문서 한 건 트랜잭션 저장 서비스 구현
9. 소규모 DB 적재와 원문 위치 복원 검증
10. 전체 3,147개 XML 적재
11. 검색 청크 생성
12. HTML 파서 별도 구현

대형·중첩 구조 회귀 테스트 후보로 다음 문서를 우선 사용할 수 있다.

| 목적           | 접수번호             | 특징                       |
|--------------|------------------|--------------------------|
| 최대 블록·표·행 수준 | 20260324000835   | 섹션 70, 블록 7,544, 표 4,872 |
| 최대 셀 수준      | 20240502000081   | 표 셀 98,995               |
| 대형 파일 성능     | 20260331004244   | 약 30MB, 최대 처리시간 6.476초   |
| 기존 문법 보정 회귀  | 보정 카운트가 있는 대표 문서 | &, <, 속성 따옴표 검증          |

## 21. 관련 코드

XML 공통 입력

```
backend/src/main/java/com/foiolens/backend/disclosure/infrastructure/xml/
├─ DartXmlInputFactoryProvider.java
├─ DartXmlSanitizingReader.java
└─ DartXmlSourceFileValidator.java
```

파서와 결과 모델

```
backend/src/main/java/com/foliolens/backend/disclosure/infrastructure/parsing/
├─ DartXmlDisclosureParser.java
├─ ParsedDisclosureDocument.java
├─ ParsedDisclosureSection.java
├─ ParsedDisclosureBlock.java
├─ ParsedDisclosureBlockType.java
├─ ParsedDisclosureTable.java
├─ ParsedDisclosureTableRow.java
├─ ParsedDisclosureTableCell.java
├─ ParsedDisclosureTableCellType.java
└─ ParsedDisclosureImage.java
```

## 배치 검증

```
backend/src/main/java/com/foliolens/backend/disclosure/infrastructure/parsing/vali
dation/
├─ XmlParsingValidationBatchRunner.java
├─ XmlParsingValidationBatchService.java
├─ XmlParsingValidationBatchResult.java
├─ XmlParsingValidationMetrics.java
├─ XmlParsingValidationMetricsCollector.java
├─ XmlParsingValidationRow.java
├─ XmlParsingValidationStatus.java
└─ XmlParsingValidationReportWriter.java
```

## 테스트

```
backend/src/test/java/com/foliolens/backend/disclosure/infrastructure/parsing/
└─ DartXmlDisclosureParserTest.java
```

## 검증 결과

```
reports/
├─ xml-parsing-validation-page-000.csv
├─ xml-parsing-validation-page-001.csv
├─ xml-parsing-validation-page-002.csv
├─ xml-parsing-validation-page-003.csv
├─ xml-parsing-validation-page-004.csv
├─ xml-parsing-validation-page-005.csv
└─ xml-parsing-validation-page-006.csv
```

## 22. 완료 체크리스트

- ☒ DART XML 전체 문서 수 확인

- ☒ 페이지 0~6 실행
- ☒ CSV UTF-8 BOM 확인
- ☒ CSV 헤더와 열 순서 확인
- ☒ 성공·경고·실패 집계
- ☒ 중복 문서 ID 확인
- ☒ DB 문서 ID와 누락·추가 비교
- ☒ 구조 조사 결과와 섹션·표 구조 비교
- ☒ 음수·역전·빈 결과 불변조건 확인
- ☒ 대형 문서 처리시간 확인
- ☒ 현재 완료 범위와 제외 범위 기록
- ☐ 파싱 결과 DB 모델 확정
- ☐ 파싱 결과 소규모 DB 적재
- ☐ 파싱 결과 전체 DB 적재
- ☐ 검색 청크 생성
- ☐ HTML·PDF 파서 구현